

Arrays in C++ have a number of drawbacks that make them inconvenient to use

- The size of the array must be a fixed constant, not a variable that you compute or read in from some input stream. If you want to read data from a file and store it in an array, you must make the array large enough to hold whatever amount of data you might ever conceivably need.
- Passing an array as a parameter to a function is inconvenient; you must also pass the size of the array as a separate parameter.
- There is no way to conveniently insert elements at the beginning or in the middle of an array. To do this, you must manually shift all of the elements over by one position to make a “hole,” and then place the new element in that hole.
- Similarly, it is inconvenient to remove an element from an array; you again have to shift all of the elements after it over by one position to fill in the gap left by the missing element.
- You cannot return an array from a function

Vectors

- Many of the problems discussed on the previous slide can be worked around by using advanced techniques, but those are beyond the scope of this course.
- Fortunately, many of these advanced techniques have been pre-packaged into a class called a vector, which we can use without having to understand all of the underlying techniques.
- Vectors, like strings, are a class in C++. Like we did with strings, we will ignore the object-oriented nature of vectors and simply note that they are another type that you can use to declare variables.

```
#include <vector>      // you must include this header
using namespace std;  // like everything else, vectors
                      // live in the std namespace
```

Advantages of Vectors

Like an array, a vector is a sequence of elements of the same type, but they provide many advantages over arrays:

- The size of a vector does not have to be a fixed constant, and it can also grow or shrink during execution. If you want to read data from a file and store it in a vector, you can just keep adding new elements as you receive them, without having to know what the largest amount would be.
- A vector always knows its own size, so passing one to a function does not require you to separately pass this information.
- Elements can be accessed by position in a vector just as they are in an array, but you can also insert and remove elements anywhere in a vector.
- If a “hole” needs to be made for an inserted element or closed for a removed element, the rest of the elements are automatically shifted to make or fill the hole.
- Vectors can be returned from a function easily

Declaring a Vector

Consider the syntax for declaring an array of 20 ints

```
const int SIZE = 20;  
  
int Numbers[SIZE];
```

The corresponding declaration for a vector of 20 ints is

```
const int SIZE = 20;  
vector<int> Numbers(SIZE);
```

Unlike an array, the elements of a vector are initialized with appropriate default values. This means 0 for ints, 0.0 for doubles, and "" for strings.

Notice that the type of elements in the vector is written in angle brackets after the name vector. This is a template in C++. Just as functions take values as parameters, some types like vector take other types as parameters. Using vector by itself would be incomplete; you must include the element type, and the combination of the template and its parameters form a unique type. For example, vector<int> and vector<double> are two distinct types

Initializing a Vector

We can also pass an optional second parameter when we declare a vector, which denotes the value that should be placed in every element of the vector. The value must be of the same type as the vector's element type, or be implicitly convertible to it.

```
const int SIZE = 5;  
vector<int> Numbers(SIZE);  
  
const int SIZE = 5;  
vector<int> Numbers(SIZE, 18);  
  
const int SIZE = 5;  
vector<double> Numbers(SIZE, 3.5);
```

0	0	0	0	0
18	18	18	18	18
3.5	3.5	3.5	3.5	3.5

Accessing a Vector

Vectors provide a size() method that reports the number of elements currently in the vector. Accessing elements by their position in a vector uses exactly the same syntax as accessing elements of an array:

```
for (size_t i = 0; i < Vec.size(); i++) {  
    cout << "Vec[" << i << "] = " << Vec[i] << endl;  
}
```

Range Bound Checking

Indexing vector locations beyond the bounds of the vector can lead to the same memory errors as in arrays.

▣The vector class provides a function, `at()`, to access a vector location that automatically checks that the memory is within the bounds of the vector:

```
for (size_t i = 0; i < Vec.size(); i++) {  
    cout << "Vec[" << i << "] = " << Vec.at(i) << endl;  
}
```

If an index passed to the vector is outside the bounds the `at` function throws an exception error that can be used to debug the violation.

Vectors Without an Initial Size

We can also declare a variable without an initial size:

```
vector<int> V;
```

When we do this, the vector is empty. It has no elements, so you can't even access `V[0]`. Calling the `size()` method will return 0. There is also a method named `empty()` that returns true if the vector is empty or false if it is not.

An empty vector doesn't seem to be very useful. Fortunately since it can grow and shrink arbitrarily, we have other ways of inserting data into it.

Resizing a Vector Explicitly

```
V.resize(int N)
```

Resizes the vector `V` so that it contains exactly `N` elements. If `V` previously had less than `N` elements (thus making it larger), all of the old elements will remain as they were. If `V` previously had more than `N` elements (thus shrinking it), any elements at index `N` or greater will be removed.

```
V.clear()
```

Removes all of the elements from the vector and sets its size to 0.

```
vector<int> V; // V starts out empty  
V.resize(10); // can now access V[0]...V[9]  
V[9] = 35;  
V.resize(9); // can now only access up to V[8]  
// The 35 that we set previously no  
// longer exists  
V.clear(); // Now V is empty again  
bool e = V.empty(); // e == true
```

Adding Items to a Vector

```
V.push_back(Type element)
```

Adds an element to the end of the vector, increasing its size by 1. The `Type` of the argument is the element type used when declaring the vector.

```
vector<int> V; // V starts out empty  
int size = V.size(); // size == 0  
V.push_back(10); // Insert 10 onto the end of V
```

```

size = V.size(); // size == 1
V.push_back(20); // Insert 20 onto the end of V
size = V.size(); // size == 2
int Foo = V[0] + V[1];

```

Inserting elements in the middle of the vector

```

#include<iostream>
#include<string>
#include<vector>
using namespace std;
int main()
{
    vector<int> test1;
    test1.push_back(1);
    test1.push_back(2);
    test1.push_back(3);
    test1.insert(test1.begin() + 1, 20);
    for (int i = 0; i < test1.size(); i++)
    {
        cout << test1[i];
    }
    system("pause");
    return 0;
}

```

Printing all elements inside a vector

```

#include<iostream>
#include<string>
#include<vector>
using namespace std;

int main()
{
    vector<int> a(9);
    a[0] = 22;
    a[1] = 23;
    for (int i = 0; i < a.size(); i++)
    {
        cout << a[i];
    }
    system("pause");
    return 0;
}

```

Copying two vectors using constructors

```

#include<iostream>
#include<string>
#include<vector>

using namespace std;
int main()
{
    vector<int> test1(3, 100);
    vector<int> test2(test1);
    for (int i = 0; i < test1.size(); i++)
    {
        cout << test2[i];
    }
}

```

```

    }
    system("pause");
    return 0;
}

```

Copying one vector to another

```

#include<iostream>
#include<string>
#include<vector>

using namespace std;
int main()
{
    vector<int> test1(3, 100);
    vector<int> test2;
    test2 = test1;
    for (int i = 0; i < test2.size(); i++)
    {
        cout << test2[i];
    }

    system("pause");
    return 0;}

```

Comparing two vectors

```

#include<iostream>
#include<string>
#include<vector>

using namespace std;
int main()
{
    vector<int> test1(3, 100);
    vector<int> test2(test1);
    if (test1 == test2)
    {
        cout << "Both the vectors are equal";
    }
    else
    {
        cout << "Both Vectors are not equal";
    }
    system("pause");
    return 0;
}

```

Removing elements from a vector

```

#include<iostream>
#include<string>
#include<vector>
using namespace std;
int main()
{
    vector<int> test1;
    test1.push_back(1);
    test1.push_back(2);
    test1.push_back(3);
    //deleting element 2
    test1.erase(test1.begin() + 1);
    for (int i = 0; i < test1.size(); i++)

```

```

        {
            cout << test1[i];
        }
        system("pause");
        return 0;
    }

```

Removing elements using pop back

```

#include<iostream>
#include<string>
#include<vector>
using namespace std;
int main()
{
    vector<int> test1;
    test1.push_back(1);
    test1.push_back(2);
    test1.push_back(3);
    //deleting element 2
    test1.erase(test1.begin() + 1);
    // removing element 3 using pop up
    test1.pop_back();
    for (int i = 0; i < test1.size(); i++)
    {
        cout << test1[i];
    }
    system("pause");
    return 0;
}

```

Iterators

Vectors provide the concept of an iterator, which is another form of representing the position of an element in the sequence.

Recall that elements in a vector of size N are identified by their index, an integer between 0 and N - 1. Iterators implement a similar concept.

The reason for this distinction is that the standard library provides other containers as well. We will see some other containers later, but in short, a container is an object that can hold some set or sequence of other objects or values.

Some containers, like vector, allow us to access their elements by numerical position. Other containers (like list, set, and map) do not. Regardless of these differences, all containers provide iterators that give us the ability to examine every element in the container, and to identify some abstract notion of an element's location in the container. At the very least, iterators allow us to start at the beginning of a container and advance from one element to the next until we have covered them all and reached the end.

```

#include<iostream>
#include<string>
#include<vector>
using namespace std;

int main()
{

```

```

vector<int> a(9);
a[0] = 22;
a[1] = 23;
vector<int>::iterator iter;
for (iter = a.begin(); iter != a.end(); iter++)
{
    cout << *iter;
}
system("pause");
return 0;
}

```

Passing vector to a function

```

#include<iostream>
#include<string>
#include<vector>
#include<algorithm>
using namespace std;
void display(vector<int> k)
{
    for (int i =0; i < k.size(); i++)
    {
        cout << k[i];
    }
}
int main()
{
    vector<int> test;
    test.push_back(1);
    test.push_back(8);
    test.push_back(4);
    display(test);
    system("pause");
    return 0;
}

```

Multidimensional Vectors

Let's create a two dimensional vector as given below

1	2	3
4	5	6

```

#include<iostream>
#include<string>
#include<vector>
using namespace std;
int main()
{
    vector<vector<int>> matrix;
    vector<int> rows;
    rows.push_back(1);
    rows.push_back(2);
    rows.push_back(3);
    matrix.push_back(rows);
    rows.clear();
    rows.push_back(4);
    rows.push_back(5);
    rows.push_back(6);
    matrix.push_back(rows);
    for(int i = 0; i < matrix.size();i++)
    {

```

```

        for (int j = 0; j < matrix[i].size(); j++)
        {
            cout << matrix[i][j];
        }
        cout << "\n";
    }

    system("pause");
    return 0;
}

```

Multidimensional vectors using iterators

```

#include<iostream>
#include<string>
#include<vector>
using namespace std;
int main()
{
    vector<vector<int>> matrix;
    vector<int> rows;
    vector<vector<int>>::iterator i;
    vector<int>::iterator j;
    rows.push_back(1);
    rows.push_back(2);
    rows.push_back(3);
    matrix.push_back(rows);
    rows.clear();
    rows.push_back(4);
    rows.push_back(5);
    rows.push_back(6);
    matrix.push_back(rows);

    for (i = matrix.begin(); i != matrix.end(); i++)
    {
        for (j = (*i).begin(); j != (*i).end(); j++)
        {
            cout << *j;
        }
        cout << "\n";
    }
    system("pause");
    return 0;
}

```

MAPS

```

#include<iostream>
#include<string>
#include<map>
using namespace std;

int main()
{
    map<string, char> grade_list;
    grade_list["satish"] = 'A';
    grade_list["Sam"] = 'B';
    map<string, char>::iterator map_it;
    map_it = grade_list.begin();

    cout << map_it->first;
}

```



```

        cout << map_it->second;
        map_it++;
        cout << map_it->first;
        cout << map_it->second;
        system("pause");
        return 0;
}

```

Updating an existing record

```

#include<iostream>
#include<string>
#include<map>
using namespace std;

int main()
{
    map<string, char> grade_list;
    grade_list["satishcj"] = 'A';
    grade_list["Sam"] = 'B';
    grade_list["satishcj"] = 'F'; // This will update the grade of Satish from A to
F
    map<string, char>::iterator map_it;
    map_it = grade_list.begin();
    cout << map_it->first;
    cout << map_it->second;
    map_it++;
    cout << map_it->first;
    cout << map_it->second;
    system("pause");
    return 0;
}

```

Printing all key and values in the map using iterator

```

#include<iostream>
#include<string>
#include<map>
using namespace std;

int main()
{
    map<string, char> grade_list;
    grade_list["Satish"] = 'A';
    grade_list["Sam"] = 'B';
    grade_list["Rahul"] = 'F';
    map<string, char>::iterator map_it;
    for (map_it = grade_list.begin(); map_it != grade_list.end(); map_it++)
    {
        cout << (*map_it).first << (*map_it).second<<"\n";
    }
    system("pause");
    return 0;
}

```

Searching for a Value in the Map

```

#include<iostream>
#include<string>

```

```

#include<map>
using namespace std;

int main()
{
    map<string, char> grade_list;
    grade_list["Satish"] = 'A';
    grade_list["Sam"] = 'B';
    grade_list["Rahul"] = 'F';
    map<string, char>::iterator map_it;
    for (map_it = grade_list.begin(); map_it != grade_list.end(); map_it++)
    {
        if((*map_it).first=="Satish")
        {
            cout << (*map_it).first << (*map_it).second << "\n";
        }
    }
    system("pause");
    return 0;
}

```

Using the Find function for searching a value in the map

```

#include<iostream>
#include<string>
#include<map>
using namespace std;

int main()
{
    map<string, char> grade_list;
    grade_list["satish"] = 'A';
    grade_list["Sam"] = 'B';
    grade_list["satish"] = 'F';
    map<string, char>::iterator map_it;
    if (grade_list.find("satishcj") == grade_list.end())
    {
        cout << "Satish cj is not in the map";
    }
    else
    {
        cout << "Satish cj is available in the map";
    }
    system("pause");
    return 0;
}

```

Checking the size of the map

```

#include<iostream>
#include<string>
#include<map>
using namespace std;

int main()
{
    map<string, char> grade_list;
    grade_list["Satishcj"] = 'A';
    grade_list["Sam"] = 'B';
    grade_list["Rahul"] = 'F';
}

```

```

        cout << grade_list.size();
        system("pause");
        return 0;
    }

```

Checking if the map is empty

```

#include<iostream>
#include<string>
#include<map>
using namespace std;

int main()
{
    map<string, char> grade_list;
    grade_list["Satishcj"] = 'A';
    grade_list["Sam"] = 'B';
    grade_list["Rahul"] = 'F';
    if (grade_list.empty())
    {
        cout << "The map is empty";
    }
    else
    {
        cout << "The map is not empty";
    }
    system("pause");
    return 0;
}

```

Removing a key from the map using erase function

```

#include<iostream>
#include<string>
#include<map>
using namespace std;

int main()
{
    map<string, char> grade_list;
    grade_list["Satishcj"] = 'A';
    grade_list["Sam"] = 'B';
    grade_list["Rahul"] = 'F';
    grade_list.erase("Satish");
    map<string, char>::iterator map_it;
    for (map_it = grade_list.begin(); map_it != grade_list.end(); map_it++)
    {
        cout << (*map_it).first << (*map_it).second;
    }
    system("pause");
    return 0;
}

```

Removing all elements from a map

```

#include<iostream>
#include<string>
#include<map>
using namespace std;

int main()
{
    map<string, char> grade_list;

```

```

        grade_list["Satishcj"] = 'A';
        grade_list["Sam"] = 'B';
        grade_list["Rahul"] = 'F';
        grade_list.clear();
        if (grade_list.empty())
        {
            cout << "The grade list is empty";
        }
        system("pause");
        return 0;
    }
}

```

LISTs in C++

Creating an integer list and printing the size and all elements inside the list

```

#include<iostream>
#include<string>
#include<list>
using namespace std;
int main()
{
    list<int> integer_list;
    integer_list.push_front(1);
    integer_list.push_front(2);
    integer_list.push_back(4);
    integer_list.push_back(3);
    list<int>::iterator itr;
    cout << integer_list.size();
    for (itr = integer_list.begin(); itr != integer_list.end(); itr++)
    {
        cout << *itr;
    }
    system("pause");
    return 0;
}

```

Inserting an element in to the list

```

#include<iostream>
#include<string>
#include<list>

using namespace std;
int main()
{
    list<int> integer_list;
    integer_list.push_front(1);
    integer_list.push_front(2);
    integer_list.push_back(4);
    integer_list.push_back(3);
    list<int>::iterator itr;
    itr=integer_list.begin();
    integer_list.insert(++itr, 5);
    for (itr = integer_list.begin(); itr != integer_list.end(); itr++)
    {
        cout << *itr;
    }

    system("pause");
}

```

```
    return 0;
```

```
}
```

Inserting in the middle of the list

```
#include<iostream>
#include<string>
#include<list>
#include<algorithm>
#include<numeric>
using namespace std;
int main()
{
    list<int> test;
    test.push_front(1);
    test.push_front(2);
    test.push_front(3);
    list<int>::iterator itr;
    itr = test.begin();

    for (itr = test.begin(); itr != test.end(); itr++)
    {
        if (*itr == 2)
        {
            test.insert(itr, 4);
        }
    }

    for (itr = test.begin(); itr != test.end(); itr++)
    {
        cout << *itr;
    }
    system("pause");
    return 0;
}
```

Removing Elements from the list using pop front and pop back

```
#include<iostream>
#include<string>
#include<list>

using namespace std;
int main()
{
    list<int> integer_list;
    integer_list.push_front(1);
    integer_list.push_front(2);
    integer_list.push_back(4);
    integer_list.push_back(3);
    list<int>::iterator itr;
    //popping one element from the front
    integer_list.pop_front();
    //popping one element from the end
    integer_list.pop_back();
    //printing all elements inside the list

    for (itr = integer_list.begin(); itr != integer_list.end(); itr++)
    {
        cout << *itr;
    }

    system("pause");
}
```

```

        return 0;
    }

```

Checking if the list is empty ,reversing the list before printing

```

#include<iostream>
#include<string>
#include<list>
using namespace std;
int main()
{
    list<int> integer_list;
    integer_list.push_front(1);
    integer_list.push_front(2);
    integer_list.push_back(4);
    integer_list.push_back(3);
    list<int>::iterator itr;
    //checking if the list is empty
    if (integer_list.empty())
    {
        cout << "The list is empty";
    }
    else
    {
        cout << "The list is not empty";
    }
    //printing all elements inside the list
    //reversing the list
    reverse(integer_list.begin(), integer_list.end());
    for (itr = integer_list.begin(); itr != integer_list.end(); itr++)
    {
        cout << *itr;
    }

    system("pause");
    return 0;
}

```

Algorithms

Algorithm for sort in ascending

```

#include<iostream>
#include<string>
#include<vector>
#include<algorithm>
using namespace std;
int main()
{
    vector<int> test;
    test.push_back(1);
    test.push_back(8);
    test.push_back(4);
    vector<int>::iterator itr1,itr2;
    itr1 = test.begin();
    itr2 = test.end();
    sort(itr1, itr2);
    for (int i = 0; i < test.size(); i++)
    {
        cout << test[i];
    }
}

```

```

    }
    system("pause");
    return 0;
}

```

Algorithm for sort in descending

```

#include<iostream>
#include<string>
#include<vector>
#include<algorithm>
using namespace std;
bool wayToSort(int i, int j) { return i > j; }
int main()
{
    vector<int> test;
    test.push_back(1);
    test.push_back(8);
    test.push_back(4);
    vector<int>::iterator itr1,itr2;
    itr1 = test.begin();
    itr2 = test.end();
    sort(itr1, itr2,wayToSort);
    for (int i = 0; i < test.size(); i++)
    {
        cout << test[i];
    }
    system("pause");
    return 0;
}

```

Searching a Vector using Algorithm

```

#include<iostream>
#include<string>
#include<vector>
#include<algorithm>
using namespace std;

int main()
{
    vector<int> test;
    test.push_back(1);
    test.push_back(8);
    test.push_back(4);
    vector<int>::iterator itr;
    int k = 8;
    itr = find(test.begin(), test.end(), k);
    if (itr == test.end())
    {
        cout << "element not found";
    }
    else
    {
        cout << "element found";
    }
    system("pause");
    return 0;
}

```

Searching for an element based on certain constraint [searching if an element greater than 4 exists using find_if]

```

#include<iostream>
#include<string>
#include<vector>
#include<algorithm>
using namespace std;
bool greater(int i)
{
    return i > 4;
}
int main()
{
    vector<int> test;
    test.push_back(1);
    test.push_back(8);
    test.push_back(4);
    vector<int>::iterator itr;
    itr = find_if(test.begin(), test.end(), greater);
    if (itr == test.end())
    {
        cout << "element not found";
    }
    else
    {
        cout << "element found";
    }
    system("pause");
    return 0;
}

```

Counting number of elements in a vector

```

#include<iostream>
#include<string>
#include<vector>
#include<algorithm>
using namespace std;
int main()
{
    vector<int> test;
    test.push_back(1);
    test.push_back(8);
    test.push_back(4);
    test.push_back(1);
    vector<int>::iterator itr;
    int ct = count(test.begin(), test.end(), 1);
    cout << ct;
    system("pause");
    return 0;
}

```

Counting based on a condition [condition is number of elements > 1]

```

#include<iostream>
#include<string>
#include<vector>
#include<algorithm>
using namespace std;
bool greater(int i)
{
    return i > 1;
}

```



```

}
int main()
{
    vector<int> test;
    test.push_back(1);
    test.push_back(8);
    test.push_back(4);
    test.push_back(1);
    vector<int>::iterator itr;
    int ct = count_if(test.begin(), test.end(), greater);
    cout << ct;
    system("pause");
    return 0;
}

```

Use of ACCUMULATE

```

#include<iostream>
#include<string>
#include<vector>
#include<algorithm>
#include<numeric>
using namespace std;
int main()
{
    vector<int> test;
    test.push_back(1);
    test.push_back(8);
    test.push_back(4);
    test.push_back(1);
    vector<int>::iterator itr;
    int sum = accumulate(test.begin(), test.end(), 0.0);
    cout << sum;
    system("pause");
    return 0;
}

```